

AI ENGINEERING SERIES | CHAPTER 5

Multi-Agent Systems & Multi-Agent RAG

Architecture • Orchestration • Routing • State Management • Conflict Resolution • MA-RAG

This chapter covers the most advanced architectural pattern in modern GenAI engineering: **Multi-Agent Systems (MAS)** and their application to **Retrieval-Augmented Generation (MA-RAG)**. Where single-agent, single-LLM architectures hit a ceiling on complexity and reliability, multi-agent systems break through by distributing work across teams of specialised AI agents — each with its own role, tools, memory, and communication protocol — coordinated by a powerful orchestration layer.

What You Will Learn:

What a multi-agent system is and why it dramatically outperforms single-agent architectures for complex tasks

Orchestrator vs. Peer-to-Peer architectural patterns — when to use each

How to implement agents as Python classes with system prompts, tools, and serialisation

The three orchestration patterns: Sequential, Parallel, and Conditional Branching

Routing strategies: Content-Based, Round-Robin, and Priority-Based routing

State management: ephemeral vs. persistent state, and the five failure-handling strategies

State coordination: synchronisation, concurrency control, and conflict resolution

Multi-Agent RAG (MA-RAG): specialist retrieval agents, coordinator, and synthesis agent

Production Python code for all concepts — fully annotated notebook cells

SECTION 01 | Theory & Concepts

Section 1: Theory — Multi-Agent Systems in Depth

1.1 What Is a Multi-Agent System?

A multi-agent system (MAS) is a computational framework in which multiple autonomous AI agents — each with a distinct role, set of capabilities, and decision-making logic — collaborate to handle complex, multi-step tasks that no single agent could address effectively on its own.

The key insight is that a multi-agent system approximates the division of labour that occurs in human organisations. Just as a hospital has radiologists, surgeons, anaesthesiologists, and nurses — each a specialist who contributes to patient outcomes without needing to know every other discipline — a MAS assigns specialised agents to distinct sub-problems, allowing the overall system to tackle questions of far greater complexity and breadth.

The Restaurant Analogy:

Host Agent: Greets incoming requests and manages routing to the right specialist.

Waiter Agent: Takes the user's order (query) and translates it into structured tasks.

Chef Agents: Specialist workers — one handles database queries, another handles web retrieval, another handles calculations.

Manager / Orchestrator Agent: Oversees the entire operation, ensuring tasks are executed in the right order, errors are handled, and the customer receives a complete, coherent response.

This mental model scales to virtually any business domain: finance, healthcare, legal, e-commerce, scientific research.

1.2 Why Multi-Agent Systems? The Limitations of Single Agents

A single LLM agent, no matter how powerful, has fundamental limitations that multi-agent architecture addresses:

Single-Agent Limitation	Multi-Agent Solution
Context window ceiling: Complex tasks require more information than fits in one prompt	Distribute subtasks across agents — each operates within its own context window
Hallucination on unknown domains: One model cannot be expert in everything	Specialised agents are fine-tuned or prompt-engineered for specific domains (SQL, medical, legal)
No parallelism: Sequential processing creates bottlenecks on independent subtasks	Parallel execution: Independent agents work simultaneously, reducing total latency by orders of magnitude

Monolithic failure: One mistake corrupts the entire response	Isolated agent failures are caught by the orchestrator; fallback agents handle errors without full system failure
Opaque reasoning: Hard to audit which knowledge source produced which claim	Specialist agents cite their sources; the synthesis agent attributes each claim to its origin agent

1.3 Key Vocabulary: Multi-Agent Systems

Term	Definition
Architecture	The fundamental organisation of a system — its components, their relationships, the communication rules, and the design principles that govern how it evolves.
Agent	An autonomous software entity with a defined role, a system prompt, a set of tools, and the ability to reason and take actions in response to inputs.
Orchestrator	A central coordinator agent that receives the user's high-level request, decomposes it into subtasks, delegates those tasks to worker agents, and aggregates the results.
Worker Agent	A specialist agent that executes a specific type of subtask (e.g., database query, document retrieval, sentiment analysis) as directed by the orchestrator.
System Prompt	A detailed natural language instruction provided to an agent that defines its persona, responsibilities, tools, and constraints. The blueprint of the agent's identity.
Tool	A Python function that an agent can invoke to perform actions beyond language processing — database queries, API calls, file I/O, web search, calculations, etc.
Serialisation	Converting a Python data object (dict, dataclass) into a transmittable string (typically JSON) for communication between agents.
Deserialisation	The reverse of serialisation: converting a received JSON string back into a Python data object the receiving agent can work with.
State	The information a system stores about the current status of a task or conversation — what has happened, what is pending, and what results have been collected so far.
Routing	The process of determining which agent (or group of agents) should receive a given message or task based on content, priority, or load-balancing rules.

1.4 Architectural Patterns: Orchestrator vs. Peer-to-Peer

The most fundamental architectural decision in a multi-agent system is how agents are organised relative to each other. There are two canonical patterns, and most real-world systems are hybrids:

Pattern 1: Orchestrator (Centralised / Hub-and-Spoke)

Mental model: A project manager who receives all incoming work, assigns it to specialists, and consolidates deliverables.

In the orchestrator pattern, a single coordinator agent receives the user's request, decomposes it into subtasks, and delegates each subtask to a worker agent with the appropriate specialisation. Worker agents do not communicate directly with each other — all communication flows through the orchestrator.

Orchestrator Pattern — E-commerce Return System Example:

1. Customer submits return request → Orchestrator receives it.
 2. Orchestrator → Policy Agent: 'Check if order #TH-9923 is within the 30-day return window.'
 3. Orchestrator → Inventory Agent: 'Prepare stock update for SKU-Y2-THR.'
 4. Orchestrator → Refund Agent: 'Process refund of \$149.99 to card ending 4242.'
 5. Orchestrator → Communication Agent: 'Send confirmation email to customer.'
 6. Orchestrator aggregates all results and returns a unified response to the customer.
- Key advantage: The orchestrator enforces the correct execution ORDER and manages shared STATE — critical for multi-step transactional workflows.

Pattern 2: Peer-to-Peer (Decentralised / Flat)

Mental model: A team of expert colleagues who pass work directly to whoever is best equipped for the next step.

In the peer-to-peer pattern, agents can communicate directly with each other. An agent that completes its task decides for itself which peer agent to hand off to next. There is no central coordinator enforcing order.

Peer-to-Peer Pattern — When to Use:

Dynamic workflows: The exact sequence of steps is not known upfront and may vary based on intermediate results.

High autonomy: Each agent is capable of routing decisions without central oversight.

Reduced bottleneck: No single orchestrator becomes a performance bottleneck under high load.

Trade-off: Harder to debug, audit, and guarantee correct execution order. State management becomes the responsibility of each individual agent.

Hybrid approach (most common in production): Use an orchestrator for top-level workflow management, but allow peer-to-peer communication between agents within the same specialist domain.

1.5 Implementing Agents: System Prompts, Tools, and Serialisation

System Prompts: The Agent's Job Description

The system prompt is the most important configuration element of any agent. It defines the agent's entire identity, scope, and behaviour. A well-crafted system prompt explicitly states:

- **Who you are:** e.g., 'You are a Data Validation Agent responsible for schema compliance checking.'
- **What you do:** e.g., 'Your sole purpose is to verify that incoming JSON payloads conform to the defined product schema.'
- **What you do NOT do:** e.g., 'You do not perform calculations, access external APIs, or modify data directly.'
- **What tools you have:** e.g., 'You ONLY have access to the `validate_schema()` and `get_schema_definition()` tools.'
- **Output format:** e.g., 'Always respond with a JSON object containing `validation_passed` (bool) and `errors` (list of strings).'

Common System Prompt Mistakes to Avoid:

Being too vague: 'Be helpful' gives the agent no actionable boundaries.

Forgetting to specify what NOT to do: Agents will attempt anything not explicitly prohibited.

Not specifying output format: Inconsistent output formats break the downstream agent that parses the response.

Omitting tool names: The LLM reasons about tools by name and description — missing either causes incorrect tool selection.

Using contradictory instructions: 'Be brief but comprehensive' creates ambiguity. Be precise and unambiguous.

Tools: Giving Agents the Ability to Act

Tools are Python functions that extend an agent's capabilities beyond language processing. Without tools, an agent can only reason about information it already knows or was given in its context. Tools allow agents to interact with the real world:

- **Database tools:** `execute_sql()`, `get_customer_record()`, `update_order_status()`
- **API tools:** `call_payment_gateway()`, `send_email()`, `query_weather_api()`
- **Retrieval tools:** `search_vector_store()`, `query_knowledge_base()`, `web_search()`
- **Calculation tools:** `compute_refund_amount()`, `calculate_shipping_cost()`

The **tool name** and **tool description** are critical metadata. When the LLM decides which tool to call, it reads the description to understand the tool's purpose and the parameters to understand how to call it. Poor descriptions lead to incorrect tool selection — a common source of agent failures in production.

Serialisation and Deserialisation

Agents in a multi-agent system frequently exchange complex data structures — Python dicts, lists, dataclasses, Pydantic models. These objects cannot be transmitted directly over a network or stored in a message queue. Serialisation converts them to a transmittable string (typically JSON); deserialisation reconstructs the object on the receiving end.

Serialisation in Practice:

Most modern agent frameworks (LangChain, AutoGen, CrewAI, Semantic Kernel) handle serialisation automatically.

You must still understand it for debugging: if Agent A sends malformed JSON, Agent B's deserialisation will fail.

Always validate serialised payloads against a schema (Pydantic models are excellent for this).

Log both the serialised outbound message and the deserialised inbound message at every agent boundary — this creates a complete audit trail.

1.6 Orchestration Patterns: Sequential, Parallel, and Conditional

Once you have built individual agents, the orchestrator must decide how to coordinate them. There are three fundamental orchestration patterns that can be combined to build any workflow:

Pattern 1: Sequential Execution

Tasks are executed one after another, where each step uses the output of the previous step as its input. This is the appropriate pattern when there is a strict data dependency between steps.

- **Example:** Customer support pipeline → Diagnostic Agent → Solution Agent → Verification Agent → Communication Agent.
- **When to use:** When Step B requires the output of Step A as input, and Step C requires Step B's output — i.e., a strict data dependency chain.
- **Trade-off:** Total latency = sum of all individual step latencies. Cannot parallelise steps even when they are logically independent.

Pattern 2: Parallel Execution

Multiple independent tasks are launched simultaneously. The orchestrator waits for all parallel tasks to complete before aggregating their results and proceeding.

- **Example:** Market analysis report → simultaneously run News Agent + Stock Data Agent + Competitor Agent, then Synthesis Agent combines all three outputs.
- **When to use:** When subtasks are independent (output of Task A is not required by Task B). Dramatically reduces total latency.
- **Implementation:** Python `asyncio.gather()` for async parallel execution, or `concurrent.futures.ThreadPoolExecutor` for thread-based parallelism.
- **Trade-off:** Requires careful state management to collect and merge results from all parallel branches without race conditions.

Pattern 3: Conditional Branching

The orchestrator evaluates the output of an agent and routes execution to different downstream agents based on the result.

- **Example:** Return system → Policy Agent returns 'Eligible' → route to Inventory Agent + Refund Agent. Policy Agent returns 'Ineligible' → route to Denial Notification Agent.
- **When to use:** When the workflow path depends on intermediate outcomes that cannot be known in advance.
- **Implementation:** A simple Python `if/elif/else` block in the orchestrator's `run()` method, evaluating agent output and calling the appropriate next agent.

1.7 Routing and Data Flow Management

Routing determines which agent receives each message or task. Data flow management ensures the message payload is in the correct format for the receiving agent.

Routing Strategies

Strategy	Description & Use Case
Content-Based Routing	Inspects the content or metadata of the message to decide which agent handles it. Example: an incoming support ticket classified as 'billing' routes to the Billing Agent; 'technical' routes to the Technical Agent. Implemented using keyword extraction, classification models, or a router LLM.
Round-Robin Routing	Distributes tasks evenly across a pool of identical agent instances. Used for load balancing when you have multiple instances of the same agent type running in parallel. Example: 10 instances of the Document Processing Agent share the workload equally.
Priority-Based Routing	Routes tasks based on a predefined priority level. High-priority tasks (e.g., security alerts, escalated customer complaints) are processed immediately; routine tasks enter a standard queue. Requires a priority queue data structure (e.g., Python <code>heapq</code> or a message broker like RabbitMQ/Azure Service Bus).

Data Flow Management

Even when routing correctly identifies the right agent, the data payload may need transformation before the receiving agent can use it. Data flow management encompasses three operations:

- **Enhancement (Enrichment):** Adding related information to the payload. Example: a customer ID is received → look up and append the customer's full profile, order history, and tier level before passing to the CRM Agent.
- **Filtering:** Removing unnecessary fields to reduce token consumption and cognitive load for the receiving agent. Example: a full customer record contains 50 fields, but the Refund Agent only needs 5 — strip the rest.
- **Transformation / Reformatting:** Converting between data formats or schemas. Example: the upstream agent returns XML; the downstream agent expects JSON. The data flow layer handles the conversion transparently.

1.8 State Management: Giving Agents Memory

A stateless agent is like a customer service representative with amnesia — every interaction starts from scratch, and the agent cannot build on prior context. For complex, multi-step tasks, agents need shared memory. This is state management.

Types of State

State Type	Description & Implementation
Ephemeral State (Short-Term)	Exists only for the duration of a single conversation or task session. Implemented in memory (Python dict or dataclass). Lost when the session ends. Examples: current conversation history, task progress within a single run, intermediate agent outputs not yet written to a database.
Persistent State (Long-Term)	Saved to a database or file system, survives across multiple sessions and system restarts. Examples: user preference profiles, historical order records, resolved ticket archives, trained model checkpoints. Implemented in relational databases (PostgreSQL), NoSQL stores (Cosmos DB), or vector databases (Azure AI Search).

Failure Handling Strategies

A robust orchestrator must anticipate failure. The following five strategies form a defence-in-depth approach to graceful degradation:

Strategy	When to Apply & How It Works
Retry Logic	First line of defence for transient errors (network timeout, rate limit exceeded). Implement with exponential backoff: retry after 1s, then 2s, then 4s, then 8s. Set a maximum retry count (e.g., 3). Libraries: tenacity (Python).

Fallback / Alternative Path	If retries are exhausted, execute Plan B. Example: if the primary database query tool fails, fall back to a cached result or a simpler read-only replica.
Graceful Failure Reporting	If the workflow cannot proceed (all retries and fallbacks exhausted), terminate cleanly. Return a structured error message to the user explaining what failed and what to try next — never crash silently.
Compensating Actions (Saga Pattern)	If a later step fails after earlier steps have committed changes, undo those earlier changes. Example: refund was processed, but inventory update failed → reverse the refund. Critical for maintaining data consistency in distributed systems.
Human-in-the-Loop Escalation	For errors that are too complex, ambiguous, or high-stakes for automated handling, pause the workflow and route to a human reviewer. The system should preserve full state so the human can review context and resume the workflow after their decision.

1.9 State Coordination: Synchronisation and Conflict Resolution

When multiple agents operate concurrently on shared state, a new class of problem emerges: agents may read stale data, or two agents may simultaneously attempt to modify the same record — producing inconsistent or corrupted state. State coordination is the discipline of preventing and resolving these conflicts.

Synchronisation Strategies

- **Database-as-Source-of-Truth:** All persistent state lives in a database. Agents use database locking (SELECT FOR UPDATE) and transactions (ACID properties) to ensure consistency. The database serialises concurrent writes — no two agents can update the same record simultaneously without conflict detection.
- **State Broadcasting / Event-Driven Updates:** When an agent successfully updates shared state, it publishes an event to a message bus (e.g., Azure Service Bus, Kafka). All agents subscribed to that event are notified and update their local knowledge accordingly. Decouples agents from direct state dependencies.
- **Optimistic Concurrency Control:** Each state record carries a version number (etag or timestamp). Before committing a write, the agent checks that the version has not changed since it last read the record. If the version has changed, another agent has modified it — the write is rejected and the agent re-reads and retries.

Conflict Resolution Strategies

- **Predefined Rules / Policies:** Embed conflict resolution logic directly in the orchestrator. Example: 'If two agents both attempt to claim the same inventory item, the agent with the earlier timestamp wins.'
- **Negotiation / Consensus:** Agents communicate directly to negotiate a resolution. Example: two competing agents submit their proposed actions to a mediator agent that evaluates both and selects the most appropriate one.

- **Rollback and Retry:** If a write fails due to a version conflict, the agent rolls back its local changes, re-reads the latest state, and retries its operation with the updated information.
- **Human Escalation:** For critical or ambiguous conflicts where automated resolution could cause irreversible harm (e.g., conflicting medical recommendations, financial transaction disputes), escalate to a human decision-maker.

1.10 Multi-Agent RAG (MA-RAG): Answering Complex Questions

Single-agent RAG (covered in Chapter 3) retrieves context from one knowledge source and generates a response. For complex queries that require synthesising information from multiple heterogeneous sources, a single retrieval agent is insufficient. Multi-Agent RAG (MA-RAG) solves this by deploying a team of specialist retrieval agents.

The Specialist Research Team Analogy:

Imagine asking: 'What are the latest developments in CRISPR for treating cystic fibrosis?'

A single agent cannot simultaneously be an expert in navigating PubMed academic databases, financial news APIs, clinical trial registries, AND write complex SQL queries against internal databases.

MA-RAG solution: Deploy specialist agents — one per knowledge source — coordinated by a Retrieval Coordinator that routes the query to the right specialists, and a Synthesis Agent that merges all retrieved knowledge into a single coherent answer.

Key Roles in a MA-RAG System

Role	Responsibility & Implementation
Specialized Retrieval Agent	An agent equipped with tools to retrieve information from ONE specific data source. Examples: PubMed Agent (uses PubMed API), SQL Agent (executes SQL queries), Web Search Agent (uses Bing/Google Search API), Vector Store Agent (queries a specific Azure AI Search index). Each agent has a system prompt defining its data source expertise.
Retrieval Coordinator (Orchestrator)	Receives the user's query, analyses it to determine which knowledge sources are relevant, and delegates retrieval sub-tasks to the appropriate specialist agents. May run multiple specialist agents in parallel. Acts as the 'research team lead.'
Synthesis Agent	Receives the retrieved information from all specialist agents and combines it into a single, coherent, well-structured answer. Attributes each claim to its source. Identifies and flags knowledge gaps. The 'report writer.'
Re-Querying / Gap Analysis Logic	A component (often implemented within the Coordinator or Synthesis Agent) that identifies missing information after the initial

	retrieval pass and triggers follow-up queries to fill those gaps before final synthesis.
--	--

The MA-RAG Pipeline: Step by Step

1. **User Query Received:** The user submits a complex, multi-faceted question.
2. **Query Decomposition:** The Retrieval Coordinator analyses the query and identifies which knowledge sources are needed. It decomposes the query into sub-queries, one per relevant data source.
3. **Parallel Retrieval:** The Coordinator dispatches sub-queries to all relevant Specialist Retrieval Agents simultaneously (parallel execution pattern).
4. **Domain-Specific Retrieval:** Each Specialist Agent uses its tools to retrieve relevant chunks from its assigned data source and returns structured results to the Coordinator.
5. **Gap Analysis:** The Coordinator (or Synthesis Agent) evaluates the retrieved results for completeness. If critical information is missing, it identifies the knowledge gap and triggers follow-up queries to the appropriate specialists.
6. **Synthesis:** The Synthesis Agent receives all retrieved chunks, combines them into a coherent answer, attributes each claim to its source, and returns the final response to the user.

SECTION 02 | Practical Implementation & Code

Section 2: Practical — Building Multi-Agent Systems in Python

2.1 Prerequisites and Setup

Requirement	Details
Python Version	Python 3.10+ (required for asyncio improvements)
Core Packages	openai>=1.0.0, python-dotenv, pydantic>=2.0, asyncio (stdlib)
Optional Packages	tenacity (retry logic), azure-search-documents (vector retrieval), httpx (async HTTP)
Environment	.env file with AZURE_OPENAI_ENDPOINT, AZURE_OPENAI_KEY, AZURE_OPENAI_API_VERSION, DEPLOYMENT_NAME

2.2 Python Notebook Cells — Complete Implementation

Cell 1 — Base Agent Class

```
# Cell 1: Foundation — the BaseAgent class that all specialist agents inherit from
# This implements the agent-as-class pattern from the architecture chapter

import os, json, asyncio
from dataclasses import dataclass, field
from typing import Any, Callable
from openai import AzureOpenAI
from dotenv import load_dotenv

load_dotenv()

# Shared Azure OpenAI client — one instance, reused by all agents
client = AzureOpenAI(
    azure_endpoint = os.getenv('AZURE_OPENAI_ENDPOINT'),
```

```
api_key      = os.getenv('AZURE_OPENAI_KEY'),
api_version  = os.getenv('AZURE_OPENAI_API_VERSION', '2024-02-15-
preview')
)
DEPLOYMENT = os.getenv('DEPLOYMENT_NAME', 'gpt-4o-deployment')

class BaseAgent:
    """
    Foundation class for all agents in the multi-agent system.
    Provides: GPT call, tool registration, serialisation, and error handling.
    """

    def __init__(self, name: str, system_prompt: str, tools: list = None):
        """
        Args:
            name          : Unique identifier for this agent (used in logging).
            system_prompt: The agent's job description - role, scope,
constraints, output format.
            tools         : List of Python callables this agent can invoke.
        """
        self.name          = name
        self.system_prompt = system_prompt
        # Tool registry: {function_name: callable}
        self.tools         = {fn.__name__: fn for fn in (tools or [])}
        # Build OpenAI tool schemas from function signatures
        self.tool_schemas = self._build_tool_schemas(tools or [])

    def _build_tool_schemas(self, tools: list) -> list:
        """
        Converts Python functions into the OpenAI function-calling schema.
        In production, use pydantic or inspect module for full type
        annotation.
        """
        schemas = []
        for fn in tools:
            schemas.append({
                'type': 'function',
                'function': {
                    'name'          : fn.__name__,
                    'description': fn.__doc__ or f'Tool: {fn.__name__}',
                    'parameters' : {
                        'type'      : 'object',
                        'properties': {}, # Simplified - use inspect for
full schema
```

```
        'required' : []
    }
}
))
return schemas

def run(self, user_message: str, context: dict = None) -> str:
    """
    Execute this agent with a user message and optional context dict.
    Handles tool calling loop automatically.

    Args:
        user_message: The task or question for this agent.
        context      : Optional dict with additional context to inject.

    Returns:
        The agent's final response as a string.
    """
    # Inject context into the user message if provided
    if context:
        context_str = json.dumps(context, indent=2)
        user_message = f'Context:\n{context_str}\n\nTask: {user_message}'

    messages = [
        {'role': 'system', 'content': self.system_prompt},
        {'role': 'user',   'content': user_message}
    ]

    print(f'[{self.name}] Processing: {user_message[:80]}...')

    # Agentic loop: keep going until the model stops calling tools
    while True:
        response = client.chat.completions.create(
            model      = DEPLOYMENT,
            messages   = messages,
            tools       = self.tool_schemas if self.tool_schemas else
None,
            tool_choice = 'auto' if self.tool_schemas else None
        )

        msg      = response.choices[0].message
        reason    = response.choices[0].finish_reason
```

```

        # If the model is done (no more tool calls), return the final
answer
        if reason == 'stop':
            print(f'[{self.name}] Done.')
            return msg.content

        # If the model wants to call a tool, execute it and loop back
        if reason == 'tool_calls' and msg.tool_calls:
            messages.append(msg) # Append the assistant's tool call
message
            for tool_call in msg.tool_calls:
                tool_name = tool_call.function.name
                tool_args = json.loads(tool_call.function.arguments)
                tool_result = self._execute_tool(tool_name, tool_args)
                # Append tool result so model can use it in next iteration
                messages.append({
                    'role': 'tool',
                    'tool_call_id': tool_call.id,
                    'content': json.dumps(tool_result)
                })

    def _execute_tool(self, name: str, args: dict) -> Any:
        """Safely execute a registered tool and return its result."""
        if name not in self.tools:
            return {'error': f'Unknown tool: {name}'}
        try:
            return self.tools[name](**args)
        except Exception as e:
            return {'error': str(e), 'tool': name, 'args': args}

```

Explanation:

BaseAgent.__init__: Stores the system prompt and converts the tools list into two forms: a dict for fast lookup during execution, and a list of OpenAI function-calling schemas for inclusion in API calls.

_build_tool_schemas(): Converts Python functions into the JSON schema format OpenAI expects for function/tool calling. In production, use `inspect.signature()` or Pydantic models to automatically extract parameter types and descriptions from function signatures.

run() — the agentic loop: The while True loop is the heart of an agent. Most tasks require multiple tool calls: the model calls a tool, gets a result, then decides to call another tool based on that result, and so on. The loop exits only when `finish_reason == 'stop'`, meaning the model has enough information to produce a final answer.

_execute_tool(): Wraps tool execution in a try/except. If a tool fails, it returns a structured error dict rather than crashing the agent. The model can then decide whether to retry, use a fallback, or inform the user of the failure.

context injection: Passing a context dict enriches the user message with state from the orchestrator — customer data, prior agent results, task parameters — without modifying the system prompt.

Cell 2 — Specialist Worker Agents

```
# Cell 2: Specialist agents — each with a focused domain and specific tools

# — Tool functions —————

def check_return_policy(order_id: str, purchase_date: str) -> dict:
    """Check if an order is eligible for return based on our 30-day policy."""
    # Simulated policy check (replace with real DB query in production)
    import datetime
    purchase = datetime.datetime.fromisoformat(purchase_date)
    days_held = (datetime.datetime.now() - purchase).days
    eligible = days_held <= 30
    return {
        'order_id' : order_id,
        'days_held' : days_held,
        'eligible' : eligible,
        'reason' : 'Within 30-day window' if eligible else f'Expired:
{days_held} days'
    }

def update_inventory(sku: str, action: str, quantity: int = 1) -> dict:
    """Update inventory levels. action: 'restock' or 'reserve'."""
    # Simulated inventory update
    return {'sku': sku, 'action': action, 'quantity': quantity,
            'status': 'success', 'new_stock_level': 47}

def process_refund(order_id: str, amount: float, payment_method: str) -> dict:
    """Process a customer refund to the original payment method."""
    return {'order_id': order_id, 'amount': amount,
            'payment_method': payment_method, 'refund_id': 'REF-20240115-
9923',
            'status': 'approved', 'processing_days': 3}

def send_customer_notification(customer_id: str, message: str,
                               channel: str = 'email') -> dict:
    """Send a notification to the customer via email or SMS."""
    return {'customer_id': customer_id, 'channel': channel,
            'status': 'sent', 'message_preview': message[:50]}
```



```
# — Specialist Agent definitions —————

policy_agent = BaseAgent(
    name          = 'PolicyAgent',
    system_prompt = """You are a Return Policy Agent.
Your sole responsibility is to verify whether a return request meets our 30-
day
return policy. You ONLY use the check_return_policy tool.
Always respond with a JSON object: {'eligible': bool, 'reason': str}."""
    tools         = [check_return_policy]
)

inventory_agent = BaseAgent(
    name          = 'InventoryAgent',
    system_prompt = """You are an Inventory Management Agent.
You update stock levels when items are returned or reserved.
You ONLY use the update_inventory tool.
Always confirm the action taken and the new stock level."""
    tools         = [update_inventory]
)

refund_agent = BaseAgent(
    name          = 'RefundAgent',
    system_prompt = """You are a Financial Refund Processing Agent.
You process approved refunds to customers' original payment methods.
You ONLY use the process_refund tool.
Always respond with the refund ID and estimated processing time."""
    tools         = [process_refund]
)

comms_agent = BaseAgent(
    name          = 'CommunicationAgent',
    system_prompt = """You are a Customer Communications Agent.
You notify customers of decisions and outcomes via their preferred channel.
You ONLY use the send_customer_notification tool.
Write clear, friendly, professional messages. Always confirm delivery."""
    tools         = [send_customer_notification]
)

print('All specialist agents initialised.')
```

 Explanation:

Tool functions: Each tool has a docstring that becomes its description in the OpenAI tool schema. Invest time in writing precise, accurate docstrings — they directly influence whether the model calls the correct tool with the correct parameters.

Agent narrowness: Notice each specialist agent has ONLY the tool relevant to its domain. This is deliberate — an agent that can access every tool is harder to reason about, harder to debug, and more likely to make mistakes. Narrowness is a feature, not a limitation.

System prompt output format: Each system prompt specifies the expected output format ('Always respond with a JSON object...'). This enables the orchestrator to parse and route results without ambiguity.

Reusability: The BaseAgent class means adding a new specialist requires only: defining the tool functions, writing the system prompt, and calling BaseAgent(). No repeated boilerplate.

Cell 3 — Orchestrator Agent with Sequential and Conditional Patterns

```
# Cell 3: Orchestrator - implements Sequential + Conditional Branching
patterns

import json
from dataclasses import dataclass, field
from typing import Optional

@dataclass
class ReturnWorkflowState:
    """
    Shared state object for the return processing workflow.
    The orchestrator updates this at each step so every agent has context.
    """
    order_id      : str
    customer_id   : str
    sku           : str
    amount        : float
    purchase_date : str
    payment_method: str
    # Results populated as workflow progresses
    policy_result : Optional[dict] = None
    inventory_result: Optional[dict] = None
    refund_result : Optional[dict] = None
    comms_result  : Optional[dict] = None
    final_status  : str = 'pending'
    errors        : list = field(default_factory=list)

class ReturnProcessingOrchestrator:
    """
```

```
Orchestrates the e-commerce return workflow.
Implements: Sequential execution + Conditional branching.
"""

def __init__(self):
    self.policy_agent = policy_agent
    self.inventory_agent = inventory_agent
    self.refund_agent = refund_agent
    self.comms_agent = comms_agent

def run(self, return_request: dict) -> ReturnWorkflowState:
    """
    Execute the full return workflow with state tracking.
    Returns the final WorkflowState with all results populated.
    """
    state = ReturnWorkflowState(**return_request)
    print(f'\n=== Starting Return Workflow for order {state.order_id}
===')

    # — STEP 1 (Sequential): Policy check —————
    try:
        policy_response = self.policy_agent.run(
            f'Check return eligibility for order {state.order_id}',
            context={
                'order_id': state.order_id,
                'purchase_date': state.purchase_date
            }
        )
        # Parse the JSON response from the policy agent
        state.policy_result = json.loads(policy_response)
    except Exception as e:
        state.errors.append(f'Policy check failed: {e}')
        state.final_status = 'error'
        return state

    # — STEP 2 (Conditional Branching): Route based on eligibility —
    if not state.policy_result.get('eligible', False):
        # BRANCH A: Ineligible → notify customer and exit
        print(f'[Orchestrator] Return INELIGIBLE:
{state.policy_result["reason"]}')
        self.comms_agent.run(
            f'Notify customer {state.customer_id} that their return was
denied.',
            context={'reason': state.policy_result['reason'], 'order_id':
state.order_id}
```

```
    )
    state.final_status = 'denied'
    return state

# BRANCH B: Eligible → proceed with inventory + refund (Sequential)
print('[Orchestrator] Return ELIGIBLE – proceeding with inventory + refund.')

# — STEP 3 (Sequential): Update inventory —————
try:
    inv_response = self.inventory_agent.run(
        f'Restock SKU {state.sku} by 1 unit.',
        context={'sku': state.sku, 'action': 'restock', 'quantity': 1}
    )
    state.inventory_result = {'status': 'done', 'detail':
inv_response}
except Exception as e:
    # Non-fatal: log error, continue to refund
    state.errors.append(f'Inventory update failed: {e}')

# — STEP 4 (Sequential): Process refund —————
try:
    refund_response = self.refund_agent.run(
        f'Process refund for order {state.order_id}.',
        context={
            'order_id'      : state.order_id,
            'amount'        : state.amount,
            'payment_method': state.payment_method
        }
    )
    state.refund_result = {'status': 'done', 'detail':
refund_response}
except Exception as e:
    state.errors.append(f'Refund processing failed: {e}')
    state.final_status = 'error'
    # COMPENSATING ACTION: If refund fails after inventory was
restocked,
    # revert the inventory update to maintain consistency
    if state.inventory_result:
        print('[Orchestrator] Compensating action: reversing inventory
update')
        self.inventory_agent.run('Reverse the restock – refund
failed.',
                                context={'sku': state.sku, 'action': 'reserve',
'quantity': 1})
    return state
```

```

# — STEP 5 (Sequential): Notify customer —————
self.comms_agent.run(
    'Send return approval confirmation to customer.',
    context={
        'customer_id': state.customer_id,
        'order_id'    : state.order_id,
        'message'     : f'Your return for order {state.order_id} has
been approved.'
                                f' Refund of ${state.amount:.2f} will arrive in
3 business days.'
    }
)

state.final_status = 'completed'
print(f'\n=== Return Workflow COMPLETED for order {state.order_id}
===')

return state

# — Run the workflow —
orchestrator = ReturnProcessingOrchestrator()
final_state = orchestrator.run({
    'order_id'      : 'TH-9923',
    'customer_id'   : 'CUST-4421',
    'sku'           : 'SKU-Y2-THR',
    'amount'        : 149.99,
    'purchase_date' : '2024-01-05',
    'payment_method': 'VISA-4242'
})
print(f'Final Status: {final_state.final_status}')
print(f'Errors: {final_state.errors}')

```

Explanation:

ReturnWorkflowState dataclass: The shared state object is the orchestrator's memory. Every step reads from and writes to this object, ensuring all agents have access to the full context of the workflow without repeatedly querying the database.

Sequential pattern: Steps 1 → 3 → 4 → 5 are sequential. Each uses the output of the previous step (Step 1's policy_result determines whether Step 3 runs at all).

Conditional branching: After Step 1, the if/else block implements the policy eligibility branch. If ineligible → BRANCH A (denial notification + exit). If eligible → BRANCH B (full return processing).

Compensating action (Saga pattern): If the refund fails AFTER inventory has already been restocked, the orchestrator reverses the inventory update. This prevents phantom stock appearing in the warehouse for a return that was never actually refunded — a critical data consistency guarantee in distributed systems.

Non-fatal vs. fatal errors: Inventory update failure is non-fatal (we continue to process the refund). Refund failure IS fatal (we cannot complete the return without the financial transaction) — it triggers compensating actions and halts.

Cell 4 — Parallel Execution with asyncio

```
# Cell 4: Parallel execution – run multiple independent agents simultaneously
# Use case: Market intelligence report requiring data from 3 independent
sources

import asyncio
import time

# — Simulated async tools (represent real API calls in production) —

async def fetch_news_analysis(company: str) -> dict:
    """Retrieve and analyse latest news articles about the company."""
    await asyncio.sleep(1.5) # Simulates API latency
    return {
        'source' : 'News',
        'company' : company,
        'sentiment': 'positive',
        'headlines': ['Q4 revenue beats estimates', 'New product launch
announced']
    }

async def fetch_stock_data(ticker: str) -> dict:
    """Pull stock price data and technical indicators."""
    await asyncio.sleep(1.2) # Simulates API latency
    return {
        'source' : 'Stock',
        'ticker' : ticker,
        'price' : 178.42,
        'pe_ratio' : 24.5,
        'trend' : 'bullish'
    }

async def fetch_competitor_data(company: str) -> dict:
    """Retrieve competitor market share and positioning data."""
    await asyncio.sleep(0.8) # Simulates API latency
```

```
    return {
        'source'      : 'Competitor',
        'company'     : company,
        'market_share': 34.2,
        'key_threats' : ['Price competition from Brand X', 'Supply chain
risk']
    }

async def parallel_research_orchestrator(company: str, ticker: str) -> dict:
    """
    Orchestrate parallel retrieval from three independent data sources.
    Total latency = max(individual latencies), NOT sum.
    """
    print(f'Launching 3 parallel retrieval agents for {company}...')
    start = time.time()

    # asyncio.gather() launches all three coroutines SIMULTANEOUSLY
    # They run in parallel – not one after another
    news_result, stock_result, competitor_result = await asyncio.gather(
        fetch_news_analysis(company),
        fetch_stock_data(ticker),
        fetch_competitor_data(company)
    )

    elapsed = time.time() - start
    print(f'All parallel agents completed in {elapsed:.2f}s '
          f'(sequential would have taken ~{1.5+1.2+0.8:.1f}s)')

    # Now run the Synthesis Agent sequentially (depends on all three results)
    synthesis_prompt = f"""
    Synthesise the following market intelligence into a 3-paragraph executive
    summary
    for {company} ({ticker}). Cite each data source.

    News Analysis: {json.dumps(news_result)}
    Stock Data: {json.dumps(stock_result)}
    Competitor Data: {json.dumps(competitor_result)}
    """

    # Use a simple GPT call for synthesis (could be a full SynthesisAgent
    instance)
    response = client.chat.completions.create(
        model      = DEPLOYMENT,
```

```

        messages = [
            {'role': 'system',
             'content': 'You are a financial analyst who synthesises data into executive summaries.'},
            {'role': 'user', 'content': synthesis_prompt}
        ]
    )

    return {
        'company' : company,
        'ticker'  : ticker,
        'raw_data' : {
            'news'      : news_result,
            'stock'     : stock_result,
            'competitor': competitor_result
        },
        'summary'  : response.choices[0].message.content
    }

# Run the parallel orchestrator
result = asyncio.run(parallel_research_orchestrator('SmartHome Corp', 'SMRT'))
print('\nExecutive Summary:')
print(result['summary'])

```

Explanation:

`asyncio.gather()`: The key function for parallel execution in Python's async ecosystem. It takes multiple coroutines and executes them concurrently. The total wait time equals the slowest coroutine (max latency), not the sum of all latencies. In this example: parallel = 1.5s, sequential = 3.5s — a 2.3x speedup.

When to use async vs. threads: `asyncio` is ideal for I/O-bound tasks (API calls, database queries, network requests) where the agent spends most of its time waiting, not computing. For CPU-bound tasks (heavy computation, image processing), use `concurrent.futures.ProcessPoolExecutor` instead.

`asyncio.run()`: Runs the top-level async coroutine in a new event loop. In a Jupyter notebook, use `await` directly instead of `asyncio.run()` (Jupyter has its own running event loop).

Hybrid sequential + parallel: The pattern here is the most common in production: parallel for independent retrieval tasks, sequential for synthesis (which depends on all retrieved data). Combining both patterns gives you both speed and correctness.

Error handling in `gather()`: If any coroutine in `gather()` raises an exception, `gather()` propagates it by default. Use `asyncio.gather(..., return_exceptions=True)` to collect exceptions as return values rather than raising them, allowing partial results to be used.

Cell 5 — Multi-Agent RAG (MA-RAG) Pipeline

```
# Cell 5: MA-RAG — Multi-Agent Retrieval Augmented Generation
# Specialist retrieval agents + coordinator + synthesis agent

from dataclasses import dataclass
import json, asyncio

@dataclass
class RetrievalResult:
    source_name : str      # Which specialist agent produced this
    content      : str      # The retrieved text chunks
    confidence   : float    # How relevant the retrieval is (0.0-1.0)
    metadata     : dict     # Source-specific metadata (URL, doc ID, query used)

# — Simulated specialist retrieval tools —————

def search_academic_database(query: str, max_results: int = 3) -> dict:
    """Search PubMed/academic database for peer-reviewed papers."""
    # Replace with real PubMed API call or Azure AI Search query
    return {
        'source' : 'PubMed',
        'query'  : query,
        'results' : [
            {'title': 'CRISPR-Cas9 delivery in lung organoids', 'abstract': 'Recent trials show...', 'year': 2024},
            {'title': 'Base editing for CF mutations', 'abstract': 'Correction of F508del...', 'year': 2023}
        ]
    }

def search_clinical_trials(condition: str, therapy_type: str) -> dict:
    """Query ClinicalTrials.gov for active trials matching condition and therapy."""
    return {
        'source' : 'ClinicalTrials.gov',
        'trials' : [
            {'trial_id': 'NCT05946863', 'phase': 'Phase 2',
             'status': 'Recruiting', 'therapy': 'CRISPR base editing for CF'},
            {'trial_id': 'NCT06012344', 'phase': 'Phase 1',
             'status': 'Active', 'therapy': 'mRNA delivery to CF lung cells'}
        ]
    }
```

```
    }

def search_news_articles(topic: str, days: int = 30) -> dict:
    """Search recent news articles on the topic from the past N days."""
    return {
        'source' : 'MedicalNewsToday',
        'articles': [
            {'headline': 'CRISPR startup raises $300M for CF treatment
program',
            'date': '2024-01-10', 'summary': 'Vertex Pharma partners
with...'}
        ]
    }

# — Specialist Retrieval Agents —————

academic_agent = BaseAgent(
    name = 'AcademicRetrievalAgent',
    system_prompt = """You are a specialist in searching academic databases
(PubMed).

Given a research query, use the search_academic_database tool to retrieve
relevant peer-reviewed papers. Return a structured summary of findings
with paper titles, key findings, and publication years.""",
    tools = [search_academic_database]
)

trials_agent = BaseAgent(
    name = 'ClinicalTrialsAgent',
    system_prompt = """You are a specialist in clinical trial databases.
Use the search_clinical_trials tool to find active trials.
Return trial IDs, phases, current status, and therapy descriptions.""",
    tools = [search_clinical_trials]
)

news_agent = BaseAgent(
    name = 'MedicalNewsAgent',
    system_prompt = """You are a specialist in medical news and industry
developments.

Use the search_news_articles tool to retrieve recent relevant news.
Summarise key developments and identify important industry trends.""",
    tools = [search_news_articles]
)
```

```
# — MA-RAG Orchestrator —————

class MARAGOrchestrator:
    """
    Multi-Agent RAG coordinator.
    Decomposes complex queries, dispatches to specialist agents,
    detects knowledge gaps, and triggers synthesis.
    """

    def __init__(self):
        self.retrieval_agents = {
            'academic'      : academic_agent,
            'clinical_trials': trials_agent,
            'news'          : news_agent
        }

    def _determine_relevant_agents(self, query: str) -> list[str]:
        """
        Use GPT to determine which specialist agents are relevant for this
        query.
        In production, this could use a router LLM or rule-based classifier.
        """
        router_response = client.chat.completions.create(
            model      = DEPLOYMENT,
            messages = [
                {'role': 'system',
                 'content': 'You are a query router. Given a research
question, '
list '
"news"]. '
                 'determine which sources are needed. Return a JSON
                 'with values from: ["academic", "clinical_trials",
                 'Return ONLY the JSON list, no other text.'},
                {'role': 'user', 'content': query}
            ],
            temperature = 0.0
        )
        try:
            return json.loads(router_response.choices[0].message.content)
        except:
            return list(self.retrieval_agents.keys()) # Default: all agents

    def _detect_knowledge_gaps(self, query: str, retrieved: dict) ->
list[str]:
        """
```

```
    Ask GPT to identify what critical information is missing from the
    retrieval.
    Returns a list of gap descriptions (empty list if no gaps).
    """
    gap_response = client.chat.completions.create(
        model      = DEPLOYMENT,
        messages = [
            {'role': 'system',
             'content': 'Identify knowledge gaps. Return a JSON list of
missing '
                        'information needed to fully answer the question.
                        'Return an empty list [] if coverage is
sufficient.'},
            {'role': 'user',
             'content': f'Question: {query}\nRetrieved:
{json.dumps(retrieved)}'}
        ],
        temperature = 0.0
    )
    try:
        return json.loads(gap_response.choices[0].message.content)
    except:
        return []

def answer(self, query: str) -> dict:
    """
    Full MA-RAG pipeline:
    1. Route query to relevant specialist agents
    2. Parallel retrieval
    3. Gap analysis and follow-up
    4. Synthesis
    """
    print(f'\n=== MA-RAG Query: {query[:70]}... ===')

    # Step 1: Determine which agents are needed
    relevant = self._determine_relevant_agents(query)
    print(f'[Coordinator] Routing to: {relevant}')

    # Step 2: Parallel retrieval from all relevant specialist agents
    retrieved = {}
    for agent_name in relevant:
        agent = self.retrieval_agents[agent_name]
        retrieved[agent_name] = agent.run(query)
        print(f'[Coordinator] {agent_name} retrieval complete.')
```

```
# Step 3: Gap analysis
gaps = self._detect_knowledge_gaps(query, retrieved)
if gaps:
    print(f'[Coordinator] Knowledge gaps detected: {gaps}')
    # Could trigger follow-up queries here – simplified for this
example

# Step 4: Synthesis
context_for_synthesis = json.dumps(retrieved, indent=2)
synthesis = client.chat.completions.create(
    model = DEPLOYMENT,
    messages = [
        {'role': 'system',
         'content': 'You are a scientific synthesis specialist. '
                    'Combine retrieved information from multiple
sources '
                    'into a comprehensive, well-structured answer. '
                    'Cite each source explicitly.'},
        {'role': 'user',
         'content': f'Question: {query}\n\nRetrieved
Information:\n{context_for_synthesis}'}
    ]
)

return {
    'query' : query,
    'agents_used' : relevant,
    'knowledge_gaps' : gaps,
    'retrieved_data' : retrieved,
    'synthesised_answer': synthesis.choices[0].message.content
}

# — Run the MA-RAG system —————
ma_rag = MARAGOrchestrator()
result = ma_rag.answer(
    'What are the latest developments in CRISPR therapy for treating cystic
fibrosis?'
)
print('\nSynthesised Answer:')
print(result['synthesised_answer'])
print(f'\nSources used: {result['agents_used']}')
print(f'Knowledge gaps: {result['knowledge_gaps']}')
```

 Explanation:

RetrievalResult dataclass: A typed container for retrieval output. Using dataclasses (or Pydantic models) instead of raw dicts enforces structure — the synthesis agent always receives results in a predictable format, preventing parsing errors.

_determine_relevant_agents(): This is the router LLM — a GPT call that classifies the query to decide which specialist agents are needed. Not every query requires all agents. Running only relevant specialists reduces cost and latency.

_detect_knowledge_gaps(): After initial retrieval, this step asks GPT to evaluate whether the retrieved information is sufficient to answer the query. If gaps are found, the coordinator can trigger follow-up queries. This 'iterative retrieval' is what distinguishes sophisticated MA-RAG from naive multi-source retrieval.

Sequential retrieval (simplified): For brevity, this example runs specialist agents sequentially. In production, use `asyncio.gather()` (Cell 4 pattern) to run all specialist agents in parallel — critical for latency when querying slow external APIs.

Synthesis with citation: The synthesis system prompt explicitly instructs the model to cite each source. This produces an auditable response where every claim is traceable to its origin agent — essential for compliance, trust, and debugging.

2.3 Quick Reference: Multi-Agent Design Decisions

Design Question	Recommended Answer
Orchestrator vs. Peer-to-Peer?	Use Orchestrator for structured workflows with defined steps. Use P2P only for highly dynamic workflows. In practice, use Orchestrator + P2P hybrid.
How many agents?	Start with the minimum needed. Each agent adds latency and complexity. Combine tasks that always run together into a single agent.
Sequential vs. Parallel?	Sequential when Step B depends on Step A's output. Parallel when steps are independent. Measure actual latency and parallelise the bottlenecks.
Ephemeral vs. Persistent state?	Ephemeral for within-session context. Persistent for anything that must survive a session end (order records, audit logs, user preferences).
How to handle tool failures?	Retry 3x with exponential backoff (tenacity library). If all retries fail, execute fallback. If no fallback, report gracefully. For critical multi-step transactions, implement compensating actions.
When to escalate to human?	Conflict resolution ambiguity, high-stakes irreversible actions, recurring automated failures after 3+ retries, explicit user request for human review.

Key Takeaways

- Multi-agent systems divide complex problems across specialised agents — each with a focused role, a narrow set of tools, and a precisely written system prompt. Narrowness is a design virtue, not a limitation.
- The Orchestrator pattern provides structured, auditable control for defined workflows. Use it as your default; adopt Peer-to-Peer only when dynamic routing is genuinely required.
- The three orchestration patterns — Sequential, Parallel, and Conditional Branching — are composable building blocks. Every complex workflow is a combination of these three primitives.
- `asyncio.gather()` enables true parallel execution. When independent subtasks each take 1–2 seconds, parallelising them can reduce total latency from $O(n)$ to $O(1)$ — one of the most impactful optimisations in multi-agent systems.
- State management is the backbone of reliability. Use the `ReturnWorkflowState` dataclass pattern: one shared state object, updated by the orchestrator at each step, accessible to all agents as context.
- Compensating actions (Saga pattern) are non-negotiable in multi-step transactional workflows. If Step 4 fails after Steps 1–3 have committed, you must undo those earlier changes to maintain data consistency.
- Multi-Agent RAG (MA-RAG) is the architecture for answering complex, multi-domain questions. Specialist retrieval agents + router LLM + gap analysis + synthesis agent = answers that no single RAG agent could produce.
- Always log serialised messages at agent boundaries, track token usage at every API call, and instrument state transitions. Observability in multi-agent systems is not optional — it is what makes debugging tractable.

Coming Up Next

Agent Evaluation & Testing | LLMOps & Monitoring | Deploying Multi-Agent Systems to Production with Azure